CustomTkinter

Guia Completo para Iniciantes

Do básico ao seu primeiro projeto completo

Conteúdo deste guia

1. O que é CustomTkinter e por que usar
2. Instalação e configuração do ambiente
3. Configurações globais e temas
4. Janela principal (CTk) e propriedades
5. Sistemas de layout: pack, grid e place
6. Widgets essenciais
7. Frames e organização visual
8. Variáveis reativas (StringVar, IntVar)
9. Imagens e ícones
10. Navegação entre telas
11. Eventos e interatividade
12. Boas práticas e estrutura de projeto
13. Projeto final completo

1. O que é CustomTkinter?

CustomTkinter é uma biblioteca Python de interface gráfica (GUI) criada por Tom Schimansky em 2021. Ela é construída em cima do Tkinter, a biblioteca padrão do Python, modernizando completamente sua aparência e adicionando novos recursos visuais.

O Tkinter original possui um visual bastante datado, com aparência dos anos 90. O CustomTkinter resolve esse problema entregando:

- Widgets com bordas arredondadas e visual moderno
- Suporte nativo a tema escuro (dark mode) e claro (light mode)
- Paletas de cores configuráveis
- Interface responsiva e escalável
- 100% compatível com o Tkinter original

> **Por que não usar o Tkinter diretamente?**
> O Tkinter ainda funciona bem, mas seu visual está muito defasado. O CustomTkinter mantém toda a simplicidade do Tkinter e adiciona um visual profissional sem exigir que você aprenda uma biblioteca completamente diferente.

Comparação com outras bibliotecas GUI

Biblioteca	Visual	Facilidade	Multiplataforma	Ideal para
CustomTkinter	Moderno	Alta	Sim	Apps desktop modernos
Tkinter	Datado	Alta	Sim	Projetos simples
PyQt6 / PySide6	Profissional	Média	Sim	Apps complexos
wxPython	Nativo do SO	Média	Sim	Apps nativos
Streamlit	Web	Alta	Sim (browser)	Dashboards de dados
Flet	Flutter	Alta	Sim	Apps multi-plataforma

2. Instalação e Configuração

Requisitos

- Python 3.7 ou superior
- pip (gerenciador de pacotes do Python)
- Tkinter (já incluso na maioria das instalações do Python)

Instalando o CustomTkinter

Abra o terminal (cmd no Windows, Terminal no macOS/Linux) e execute:

```
pip install customtkinter
```

Instalando dependências opcionais

O Pillow é recomendado para trabalhar com imagens no CustomTkinter:

```
pip install pillow
```

Verificando a instalação

Para confirmar que a instalação foi bem-sucedida, execute no Python:

```
import customtkinter as ctk
print(ctk.__version__)
```

💡 Se aparecer o número da versão sem erros, a instalação foi bem-sucedida!

Atualizando para a versão mais recente

```
pip install --upgrade customtkinter
```

3. Configurações Globais e Temas

Antes de criar qualquer janela, você pode definir o comportamento visual do app inteiro usando duas funções globais. Essas configurações afetam TODOS os widgets da aplicação.

Modo de aparência

```
import customtkinter as ctk

# Opções: 'dark', 'light', 'system'
ctk.set_appearance_mode('dark')    # Tema escuro
ctk.set_appearance_mode('light')   # Tema claro
ctk.set_appearance_mode('system')  # Segue o sistema operacional
```

Tema de cores

```
# Opções: 'blue', 'green', 'dark-blue'
ctk.set_default_color_theme('blue')      # Azul (padrão)
ctk.set_default_color_theme('green')     # Verde
ctk.set_default_color_theme('dark-blue') # Azul escuro
```

> 💡 Você pode criar temas personalizados salvando um arquivo JSON com suas cores e passando o caminho do arquivo para set_default_color_theme().

Alterando o tema em tempo de execução

É possível mudar o tema enquanto o app está rodando, por exemplo, através de um botão:

```
import customtkinter as ctk

app = ctk.CTk()

def alternar_tema():
    modo_atual = ctk.get_appearance_mode()
    if modo_atual == 'Dark':
        ctk.set_appearance_mode('light')
    else:
        ctk.set_appearance_mode('dark')

btn = ctk.CTkButton(app, text='Alternar Tema', command=alternar_tema)
btn.pack(pady=20)

app.mainloop()
```

4. A Janela Principal (CTk)

A janela principal é criada com a classe CTk(). Ela é o ponto de entrada de todo o app e deve ser criada antes de qualquer widget.

Criando e configurando a janela

```
import customtkinter as ctk

ctk.set_appearance_mode('dark')
ctk.set_default_color_theme('blue')

app = ctk.CTk()

# Título da janela
app.title('Meu Aplicativo')

# Tamanho: largura x altura em pixels
app.geometry('800x600')

# Posicionar no centro da tela (largura x altura + x + y)
app.geometry('800x600+200+100')

# Tamanho mínimo e máximo
app.minsize(400, 300)
app.maxsize(1920, 1080)

# Bloquear redimensionamento
app.resizable(False, False)  # (horizontal, vertical)

# Ícone (apenas Windows, arquivo .ico)
# app.iconbitmap('icone.ico')

# Inicia o loop de eventos - SEMPRE ao final
app.mainloop()
```

Centralizar a janela na tela

```
def centralizar_janela(app, largura, altura):
    tela_larg = app.winfo_screenwidth()
    tela_alt  = app.winfo_screenheight()
    x = (tela_larg // 2) - (largura // 2)
    y = (tela_alt  // 2) - (altura  // 2)
    app.geometry(f'{largura}x{altura}+{x}+{y}')

app = ctk.CTk()
centralizar_janela(app, 800, 600)
app.mainloop()
```

⚠️ O mainloop() deve ser sempre a última linha do código. Tudo que vier depois dele não será executado até que a janela seja fechada.

5. Sistemas de Layout

O layout define como os widgets são posicionados dentro da janela ou frame. O CustomTkinter herda os três sistemas de layout do Tkinter.

Pack — Empilhamento automático

O pack é o mais simples: empilha os widgets automaticamente em uma direção.

```
widget.pack(
    pady=10,          # Espaçamento externo vertical (pixels)
    padx=10,          # Espaçamento externo horizontal (pixels)
    fill='x',         # Preencher: 'x', 'y', 'both' ou 'none'
    expand=True,      # Expandir com a janela
    side='top',       # Direção: 'top', 'bottom', 'left', 'right'
    anchor='center',  # Ancoragem: 'n','s','e','w','center'
    ipadx=5,          # Espaçamento interno horizontal
    ipady=5,          # Espaçamento interno vertical
)
```

Grid — Posicionamento em grade

O grid permite posicionar widgets em linhas e colunas, como uma tabela. É o mais versátil para layouts complexos.

```
# Configurar peso das colunas (permite expansão proporcional)
app.grid_columnconfigure(0, weight=1)
app.grid_columnconfigure(1, weight=2)  # coluna 1 expande o dobro
app.grid_rowconfigure(0, weight=1)

widget.grid(
    row=0,           # Linha
    column=1,        # Coluna
    padx=10,         # Espaçamento externo horizontal
    pady=10,         # Espaçamento externo vertical
    sticky='ew',     # Esticar: 'n','s','e','w','ns','ew','nsew'
    columnspan=2,    # Ocupar múltiplas colunas
    rowspan=2,       # Ocupar múltiplas linhas
    ipadx=5,         # Padding interno horizontal
    ipady=5,         # Padding interno vertical
)
```

Place — Posição absoluta

O place posiciona widgets em coordenadas exatas de pixels. Menos recomendado pois não se adapta a redimensionamentos.

```
widget.place(
    x=100,             # Posição horizontal em pixels
```

```
    y=200,              # Posição vertical em pixels
    relx=0.5,           # Posição relativa (0.0 a 1.0)
    rely=0.5,           # Posição relativa vertical
    anchor='center',    # Ponto de ancoragem do widget
)
```

⚠️ NUNCA misture pack e grid no mesmo Frame ou janela — isso causa um erro fatal que trava o app.

Sistema	Uso ideal	Vantagens	Desvantagens
pack	Layouts simples, listas	Muito simples de usar	Pouco controle de posição
grid	Formulários, layouts complexos	Controle preciso com linhas/colunas	Mais verboso
place	Posições exatas, animações	Controle total de posição	Não se adapta ao redimensionamento

6. Widgets Essenciais

Widgets são os elementos visuais do app: botões, campos de texto, rótulos, etc. Abaixo estão os principais com seus parâmetros mais usados.

6.1 CTkLabel — Rótulo de texto

Exibe texto ou imagens na tela. Não é interativo.

```
label = ctk.CTkLabel(
    app,
    text='Olá, mundo!',
    font=('Arial', 18, 'bold'),   # (família, tamanho, estilo)
    text_color='white',           # Cor do texto
    fg_color='transparent',       # Cor de fundo ('transparent' = sem fundo)
    width=200,                    # Largura em pixels
    height=40,                    # Altura em pixels
    anchor='w',                   # Alinhamento: 'w','e','n','s','center'
    wraplength=300,               # Quebra de linha após N pixels
    justify='left',               # Alinhamento multi-linha
    corner_radius=8,              # Arredondamento (com fg_color)
)
label.pack(pady=10)

# Atualizar o texto em tempo de execução
label.configure(text='Novo texto')
```

6.2 CTkButton — Botão

O widget mais usado. Executa uma ação ao ser clicado.

```
def ao_clicar():
    print('Botão clicado!')

btn = ctk.CTkButton(
    app,
    text='Clique aqui',
    command=ao_clicar,           # Função chamada ao clicar
    width=200,
    height=40,
    font=('Arial', 14),
    fg_color='#1F6AA5',          # Cor do botão
    hover_color='#144870',       # Cor ao passar o mouse
    text_color='white',
    corner_radius=8,
    border_width=0,
    border_color='#FFFFFF',
    state='normal',              # 'normal' ou 'disabled'
    image=None,                  # CTkImage para ícone
    compound='left',             # Posição da imagem:
'left','right','top','bottom'
)
btn.pack(pady=10)
```

```
# Desabilitar o botão
btn.configure(state='disabled')

# Passar argumentos para o command com lambda
btn2 = ctk.CTkButton(app, text='Com args',
    command=lambda: print('Argumento:', 42))
```

6.3 CTkEntry — Campo de texto

Campo para o usuário digitar uma linha de texto.

```
entry = ctk.CTkEntry(
    app,
    placeholder_text='Digite aqui...',  # Texto de dica
    width=300,
    height=40,
    font=('Arial', 14),
    text_color='white',
    fg_color='#2b2b2b',
    border_color='#555555',
    border_width=2,
    corner_radius=8,
    show='*',                           # Ocultar texto (para senhas)
    justify='left',
    state='normal',                     # 'normal', 'disabled' ou 'readonly'
)
entry.pack(pady=10)

# Ler o valor digitado
valor = entry.get()

# Inserir texto no campo
entry.insert(0, 'Texto inicial')  # posição, texto

# Limpar o campo
entry.delete(0, 'end')
```

6.4 CTkTextbox — Área de texto

Campo para múltiplas linhas de texto. Ideal para notas, logs, descrições.

```
textbox = ctk.CTkTextbox(
    app,
    width=400,
    height=200,
    font=('Arial', 13),
    wrap='word',          # Quebra de linha: 'word', 'char', 'none'
    state='normal',       # 'normal' ou 'disabled'
    activate_scrollbars=True,
)
textbox.pack(pady=10)
```

```
# Inserir texto
textbox.insert('end', 'Linha 1\nLinha 2\n')

# Ler todo o conteúdo
conteudo = textbox.get('1.0', 'end')

# Limpar
textbox.delete('1.0', 'end')

# Deixar somente leitura
textbox.configure(state='disabled')
```

6.5 CTkCheckBox — Caixa de seleção

Permite ao usuário marcar ou desmarcar uma opção.

```
check = ctk.CTkCheckBox(
    app,
    text='Aceito os termos de uso',
    font=('Arial', 13),
    command=lambda: print('Marcado:', check.get()),
    fg_color='#1F6AA5',        # Cor quando marcado
    hover_color='#144870',
    border_color='#888888',
    checkmark_color='white',
    corner_radius=4,
    border_width=2,
)
check.pack(pady=10)

check.get()      # Retorna 1 (marcado) ou 0 (desmarcado)
check.select()   # Marcar
check.deselect() # Desmarcar
check.toggle()   # Alternar
```

6.6 CTkRadioButton — Botão de seleção única

Permite selecionar apenas uma opção entre várias. Todos os radio buttons de um grupo devem compartilhar a mesma variável.

```
# Variável compartilhada entre os botões
var_opcao = ctk.StringVar(value='op1')

r1 = ctk.CTkRadioButton(app, text='Opção 1', variable=var_opcao, value='op1')
r2 = ctk.CTkRadioButton(app, text='Opção 2', variable=var_opcao, value='op2')
r3 = ctk.CTkRadioButton(app, text='Opção 3', variable=var_opcao, value='op3')

r1.pack(pady=5)
r2.pack(pady=5)
r3.pack(pady=5)
```

```
# Ler a opção selecionada
print(var_opcao.get())  # retorna: 'op1', 'op2' ou 'op3'
```

6.7 CTkSwitch — Interruptor

Similar ao checkbox, mas com visual de interruptor on/off.

```
switch = ctk.CTkSwitch(
    app,
    text='Ativar notificações',
    font=('Arial', 13),
    command=lambda: print('Estado:', switch.get()),
    fg_color='#555555',          # Cor quando desligado
    progress_color='#1F6AA5',    # Cor quando ligado
    button_color='white',
)
switch.pack(pady=10)

switch.get()      # Retorna 1 (ligado) ou 0 (desligado)
switch.select()   # Ligar
switch.deselect() # Desligar
switch.toggle()   # Alternar
```

6.8 CTkSlider — Controle deslizante

Permite selecionar um valor dentro de um intervalo arrastando o controle.

```
def ao_mover(valor):
    print(f'Valor: {valor:.1f}')

slider = ctk.CTkSlider(
    app,
    from_=0,                # Valor mínimo
    to=100,                 # Valor máximo
    number_of_steps=10,     # Passos discretos (None = contínuo)
    orientation='horizontal', # ou 'vertical'
    command=ao_mover,
    width=300,
    fg_color='#333333',     # Cor da trilha
    progress_color='#1F6AA5', # Cor do progresso
    button_color='#1F6AA5',   # Cor do botão deslizante
)
slider.pack(pady=10)

slider.get()       # Retorna o valor atual
slider.set(50)     # Define um valor
```

6.9 CTkProgressBar — Barra de progresso

Indica o progresso de uma tarefa. Pode ser determinada (0 a 100%) ou indeterminada (animação contínua).

```
# Barra determinada
progress = ctk.CTkProgressBar(app, width=300)
progress.pack(pady=10)
progress.set(0)     # 0.0 = 0% | 1.0 = 100%
progress.set(0.75)  # 75%

# Barra indeterminada (animação contínua)
progress_ind = ctk.CTkProgressBar(
    app,
    mode='indeterminate',
    indeterminate_speed=1.0
)
progress_ind.pack(pady=10)
progress_ind.start()  # Iniciar animação
progress_ind.stop()   # Parar animação
```

6.10 CTkComboBox — Lista suspensa editável

Permite selecionar uma opção de uma lista ou digitar um valor personalizado.

```
combo = ctk.CTkComboBox(
    app,
    values=['Python', 'JavaScript', 'Rust', 'Go'],
    command=lambda v: print('Selecionado:', v),
    width=200,
    state='normal',   # 'normal' (editável) ou 'readonly' (só seleção)
)
combo.pack(pady=10)

combo.get()              # Retorna o valor atual
combo.set('Python')      # Define um valor
combo.configure(values=['A', 'B', 'C'])  # Atualizar opções
```

6.11 CTkOptionMenu — Menu de opções

Similar ao ComboBox, mas sem campo de texto editável. Apenas seleção da lista.

```
menu = ctk.CTkOptionMenu(
    app,
    values=['Iniciante', 'Intermediário', 'Avançado'],
    command=lambda v: print('Nível:', v),
    width=200,
    fg_color='#1F6AA5',
    button_color='#144870',
    button_hover_color='#0d2f4f',
)
menu.pack(pady=10)
```

```
menu.get()               # Retorna a opção selecionada
menu.set('Intermediário') # Define a opção
```

6.12 CTkSegmentedButton — Botões segmentados

Grupo de botões onde apenas um pode ser selecionado por vez. Ótimo para filtros e abas simples.

```
seg = ctk.CTkSegmentedButton(
    app,
    values=['Dia', 'Semana', 'Mês', 'Ano'],
    command=lambda v: print('Período:', v),
    selected_color='#1F6AA5',
    selected_hover_color='#144870',
)
seg.pack(pady=10)

seg.get()         # Retorna o valor selecionado
seg.set('Semana') # Selecionar um valor
```

7. Frames e Organização Visual

Frames são contêineres que agrupam outros widgets. São fundamentais para organizar layouts complexos, criar seções e separar responsabilidades visuais.

7.1 CTkFrame — Contêiner básico

```
frame = ctk.CTkFrame(
    app,
    width=300,
    height=200,
    corner_radius=10,
    fg_color='#2b2b2b',      # Cor de fundo
    border_width=2,
    border_color='#555555',
)
frame.pack(padx=20, pady=20, fill='both', expand=True)

# Adicionar widgets dentro do frame
label = ctk.CTkLabel(frame, text='Dentro do Frame')
label.pack(pady=10)
```

7.2 CTkScrollableFrame — Frame com scroll

Frame que adiciona barra de rolagem automaticamente quando o conteúdo excede o tamanho definido.

```
scroll = ctk.CTkScrollableFrame(
    app,
    width=300,
    height=400,
    label_text='Lista de Itens',   # Título opcional
    label_font=('Arial', 14, 'bold'),
    fg_color='#2b2b2b',
    scrollbar_button_color='#1F6AA5',
    orientation='vertical',        # ou 'horizontal'
)
scroll.pack(padx=20, pady=20)

# Adicionar muitos widgets — o scroll aparece automaticamente
for i in range(50):
    ctk.CTkLabel(scroll, text=f'Item {i+1}').pack(pady=2)
```

7.3 CTkTabview — Abas

Cria uma interface com abas, onde cada aba pode conter widgets diferentes.

```
tabs = ctk.CTkTabview(
    app,
```

```
    width=500,
    height=350,
    anchor='nw',         # Posição das abas: 'nw','n','ne','sw','s','se'
    fg_color='#2b2b2b',
    segmented_button_selected_color='#1F6AA5',
)
tabs.pack(pady=20, padx=20, fill='both', expand=True)

# Criar abas
tabs.add('Perfil')
tabs.add('Configurações')
tabs.add('Sobre')

# Acessar o frame de cada aba e adicionar widgets
ctk.CTkLabel(tabs.tab('Perfil'), text='Dados do Perfil').pack(pady=10)
ctk.CTkLabel(tabs.tab('Configurações'), text='Configs aqui').pack(pady=10)

# Navegar para uma aba programaticamente
tabs.set('Configurações')

# Saber qual aba está ativa
print(tabs.get())
```

7.4 Layout com Sidebar (exemplo prático)

```
import customtkinter as ctk

app = ctk.CTk()
app.geometry('700x450')
app.title('Layout com Sidebar')

# Sidebar esquerda
sidebar = ctk.CTkFrame(app, width=180, corner_radius=0)
sidebar.pack(side='left', fill='y')
sidebar.pack_propagate(False)  # Manter largura fixa

ctk.CTkLabel(sidebar, text='Menu', font=('Arial', 18, 'bold')).pack(pady=25)
ctk.CTkButton(sidebar, text='Inicio').pack(pady=5, padx=15, fill='x')
ctk.CTkButton(sidebar, text='Perfil').pack(pady=5, padx=15, fill='x')
ctk.CTkButton(sidebar, text='Config').pack(pady=5, padx=15, fill='x')
ctk.CTkButton(sidebar, text='Sair').pack(pady=5, padx=15, fill='x')

# Área principal
main = ctk.CTkFrame(app)
main.pack(side='left', fill='both', expand=True, padx=20, pady=20)

ctk.CTkLabel(main, text='Bem-vindo!', font=('Arial', 22, 'bold')).pack(pady=20)

app.mainloop()
```

8. Variáveis Reativas

As variáveis reativas são objetos especiais do Tkinter que permitem vincular valores a widgets e detectar mudanças automaticamente, sem precisar chamar .get() manualmente a todo momento.

Tipos de variáveis

Classe	Tipo de dado	Uso comum
StringVar	Texto (str)	Campos de texto, labels dinâmicos
IntVar	Inteiro (int)	Checkboxes, radio buttons numéricos
DoubleVar	Decimal (float)	Sliders, campos numéricos
BooleanVar	Booleano (bool)	Switches, checkboxes

Usando StringVar

```
import customtkinter as ctk

app = ctk.CTk()

# Criar variável reativa
nome_var = ctk.StringVar(value='')  # valor inicial

# Vincular ao Entry — label atualiza automaticamente ao digitar
entry = ctk.CTkEntry(app, textvariable=nome_var, width=300)
entry.pack(pady=10)

# Vincular ao Label — reflete o valor do Entry em tempo real
label = ctk.CTkLabel(app, textvariable=nome_var)
label.pack(pady=10)

# Monitorar mudanças com trace
def ao_mudar(*args):
    print(f'Novo valor: {nome_var.get()}')

nome_var.trace_add('write', ao_mudar)  # 'write', 'read' ou 'unset'

# Ler e definir valores manualmente
valor = nome_var.get()
nome_var.set('Novo texto')

app.mainloop()
```

9. Imagens e Ícones

O CustomTkinter usa CTkImage para exibir imagens, suportando versões diferentes para tema claro e escuro. Requer o Pillow instalado.

Carregando e exibindo imagens

```
import customtkinter as ctk
from PIL import Image

app = ctk.CTk()

# Carregar imagem com Pillow
img_pillow = Image.open('imagem.png')

# Criar CTkImage (suporte a tema claro/escuro)
imagem = ctk.CTkImage(
    light_image=Image.open('logo_dark.png'),   # Para tema claro
    dark_image=Image.open('logo_light.png'),   # Para tema escuro
    size=(100, 100)                            # Tamanho em pixels
)

# Exibir em um Label (text='' para não mostrar texto)
label_img = ctk.CTkLabel(app, image=imagem, text='')
label_img.pack(pady=10)

# Usar como ícone em um botão
btn = ctk.CTkButton(
    app,
    image=imagem,
    text='Com ícone',
    compound='left',  # Imagem à esquerda do texto
)
btn.pack(pady=10)

app.mainloop()
```

💡 Se a imagem não tiver versão clara/escura, passe a mesma imagem para light_image e dark_image. O size é obrigatório no CTkImage.

10. Navegação entre Telas

Em apps com múltiplas telas, a abordagem mais organizada é criar cada tela como um CTkFrame e alternar entre elas conforme o usuário navega.

```
import customtkinter as ctk

# ── Cada tela é um Frame ──────────────────────────────────────
class TelaInicio(ctk.CTkFrame):
    def __init__(self, master):
        super().__init__(master)
        ctk.CTkLabel(self, text='Tela Inicial',
            font=('Arial', 22, 'bold')).pack(pady=30)
        ctk.CTkButton(self, text='Ir para Login',
            command=lambda: master.mostrar('login')).pack(pady=10)


class TelaLogin(ctk.CTkFrame):
    def __init__(self, master):
        super().__init__(master)
        ctk.CTkLabel(self, text='Login',
            font=('Arial', 22, 'bold')).pack(pady=20)
        ctk.CTkEntry(self, placeholder_text='E-mail').pack(pady=8)
        ctk.CTkEntry(self, placeholder_text='Senha', show='*').pack(pady=8)
        ctk.CTkButton(self, text='Entrar',
            command=lambda: master.mostrar('inicio')).pack(pady=10)


# ── App principal gerencia as telas ───────────────────────────
class App(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title('App Multi-telas')
        self.geometry('500x400')

        # Instanciar todas as telas
        self.telas = {
            'inicio': TelaInicio(self),
            'login':  TelaLogin(self),
        }

        # Mostrar a tela inicial
        self.mostrar('inicio')

    def mostrar(self, nome):
        # Esconder todas as telas
        for tela in self.telas.values():
            tela.pack_forget()
        # Mostrar a tela desejada
        self.telas[nome].pack(fill='both', expand=True)


App().mainloop()
```

11. Eventos e Interatividade

Além do parâmetro command nos botões, o CustomTkinter/Tkinter oferece um sistema de eventos (bind) para capturar qualquer interação do usuário.

11.1 Vinculando eventos com bind()

```
# Sintaxe: widget.bind('<Evento>', funcao)

# Clicar com o mouse
label.bind('<Button-1>', lambda e: print('Clique esquerdo'))
label.bind('<Button-3>', lambda e: print('Clique direito'))
label.bind('<Double-Button-1>', lambda e: print('Duplo clique'))

# Teclado
entry.bind('<Return>', lambda e: print('Enter pressionado'))
entry.bind('<KeyPress>', lambda e: print(f'Tecla: {e.char}'))
app.bind('<Control-s>', lambda e: salvar())

# Mouse sobre o widget
label.bind('<Enter>', lambda e: label.configure(text_color='yellow'))
label.bind('<Leave>', lambda e: label.configure(text_color='white'))

# Redimensionamento da janela
app.bind('<Configure>', lambda e: print(f'{e.width}x{e.height}'))
```

11.2 Atualização com after()

Para tarefas que precisam rodar periodicamente sem travar a interface, use app.after() em vez de loops ou time.sleep().

```
import customtkinter as ctk
from datetime import datetime

app = ctk.CTk()

label_hora = ctk.CTkLabel(app, font=('Arial', 28))
label_hora.pack(pady=30)

def atualizar_hora():
    agora = datetime.now().strftime('%H:%M:%S')
    label_hora.configure(text=agora)
    app.after(1000, atualizar_hora)  # Chamar novamente após 1000ms

atualizar_hora()  # Iniciar o ciclo
app.mainloop()
```

> ⚠️ **Nunca use time.sleep() dentro da interface gráfica! Isso trava completamente o app. Use sempre app.after() para atrasos e atualizações periódicas.**

11.3 Threads para tarefas pesadas

Para tarefas demoradas (downloads, processamento), use threads para não travar a interface:

```
import customtkinter as ctk
import threading
import time

app = ctk.CTk()

progress = ctk.CTkProgressBar(app, width=300)
progress.pack(pady=20)
progress.set(0)

btn = ctk.CTkButton(app, text='Iniciar Tarefa')
btn.pack(pady=10)

def tarefa_pesada():
    btn.configure(state='disabled')
    for i in range(1, 11):
        time.sleep(0.5)              # Simula trabalho
        progress.set(i / 10)         # Atualiza a barra
    btn.configure(state='normal')

def iniciar():
    # Rodar em thread separada para não travar a UI
    thread = threading.Thread(target=tarefa_pesada, daemon=True)
    thread.start()

btn.configure(command=iniciar)
app.mainloop()
```

12. Boas Práticas

Seguir boas práticas desde o início torna o código mais legível, organizado e fácil de manter à medida que o projeto cresce.

Usar classes para organizar o código

Para qualquer app além do mais simples, organize o código em classes:

- Cada tela = uma classe herdando de CTkFrame
- O app principal = uma classe herdando de CTk
- Evite variáveis globais — passe referências pelo construtor

Estrutura de arquivos recomendada

```
meu_app/
├── main.py          # Ponto de entrada — cria e inicia o app
├── app.py           # Classe principal (herda de CTk)
├── telas/
│   ├── inicio.py    # Classe TelaInicio
│   ├── login.py     # Classe TelaLogin
│   └── config.py    # Classe TelaConfig
├── componentes/
│   ├── sidebar.py   # Componente reutilizável
│   └── card.py      # Componente reutilizável
├── assets/
│   ├── icones/      # Imagens e ícones
│   └── temas/       # Arquivos JSON de tema
└── utils/
    └── helpers.py   # Funções auxiliares
```

Resumo de boas práticas

- Sempre defina set_appearance_mode e set_default_color_theme antes de criar a janela
- Use grid_columnconfigure e grid_rowconfigure com weight=1 para layouts responsivos
- Prefira grid ao pack em layouts com múltiplas colunas
- Use pack_propagate(False) em frames com tamanho fixo
- Nunca chame mainloop() mais de uma vez — apenas no app principal
- Destrua janelas secundárias com .destroy() em vez de .withdraw() quando não forem mais necessárias
- Use after() para atualizações periódicas, nunca time.sleep()
- Use threads para operações de I/O e processamento pesado

13. Projeto Final Completo

Para colocar tudo em prática, veja abaixo um app de cadastro completo, com validação, navegação entre telas e boas práticas aplicadas.

```
import customtkinter as ctk

ctk.set_appearance_mode('dark')
ctk.set_default_color_theme('blue')

# ── Tela de Login ──────────────────────────────────────────────
class TelaLogin(ctk.CTkFrame):
    def __init__(self, master):
        super().__init__(master, fg_color='transparent')

        ctk.CTkLabel(self, text='Bem-vindo!',
            font=('Arial', 28, 'bold')).pack(pady=(40, 5))
        ctk.CTkLabel(self, text='Faça login para continuar',
            text_color='gray').pack(pady=(0, 30))

        self.entry_email = ctk.CTkEntry(
            self, width=320, height=42,
            placeholder_text='E-mail')
        self.entry_email.pack(pady=6)

        self.entry_senha = ctk.CTkEntry(
            self, width=320, height=42,
            placeholder_text='Senha', show='*')
        self.entry_senha.pack(pady=6)

        self.label_erro = ctk.CTkLabel(self, text='', text_color='#E74C3C')
        self.label_erro.pack(pady=5)

        ctk.CTkButton(self, text='Entrar', width=320, height=42,
            command=self.fazer_login).pack(pady=10)

        ctk.CTkButton(self, text='Criar conta', width=320,
            fg_color='transparent', border_width=1,
            command=lambda: master.mostrar('cadastro')).pack(pady=5)

    def fazer_login(self):
        email = self.entry_email.get()
        senha = self.entry_senha.get()
        if not email or not senha:
            self.label_erro.configure(text='Preencha todos os campos!')
        elif email == 'admin@email.com' and senha == '1234':
            self.label_erro.configure(text='')
            self.master.mostrar('dashboard')
        else:
            self.label_erro.configure(text='E-mail ou senha incorretos!')


# ── Tela de Cadastro ───────────────────────────────────────────
class TelaCadastro(ctk.CTkFrame):
    def __init__(self, master):
```

```
        super().__init__(master, fg_color='transparent')

        ctk.CTkLabel(self, text='Criar Conta',
            font=('Arial', 26, 'bold')).pack(pady=(40, 25))

        campos = [
            ('Nome completo', False),
            ('E-mail', False),
            ('Senha', True),
            ('Confirmar senha', True),
        ]
        self.entries = {}
        for placeholder, ocultar in campos:
            e = ctk.CTkEntry(self, width=320, height=42,
                placeholder_text=placeholder,
                show='*' if ocultar else '')
            e.pack(pady=5)
            self.entries[placeholder] = e

        self.check = ctk.CTkCheckBox(self, text='Aceito os termos de uso')
        self.check.pack(pady=10)

        self.label_status = ctk.CTkLabel(self, text='')
        self.label_status.pack(pady=5)

        ctk.CTkButton(self, text='Cadastrar', width=320, height=42,
            command=self.cadastrar).pack(pady=8)
        ctk.CTkButton(self, text='Voltar ao login', width=320,
            fg_color='transparent', border_width=1,
            command=lambda: master.mostrar('login')).pack(pady=4)

    def cadastrar(self):
        nome  = self.entries['Nome completo'].get()
        email = self.entries['E-mail'].get()
        senha = self.entries['Senha'].get()
        conf  = self.entries['Confirmar senha'].get()

        if not all([nome, email, senha, conf]):
            self.label_status.configure(
                text='Preencha todos os campos!', text_color='#E74C3C')
        elif senha != conf:
            self.label_status.configure(
                text='As senhas nao conferem!', text_color='#E74C3C')
        elif not self.check.get():
            self.label_status.configure(
                text='Aceite os termos!', text_color='#E74C3C')
        else:
            self.label_status.configure(
                text=f'Conta criada para {nome}!', text_color='#2ECC71')
            self.master.after(1500, lambda: self.master.mostrar('login'))


# ── Dashboard ──────────────────────────────────────────────────────
class TelaDashboard(ctk.CTkFrame):
    def __init__(self, master):
        super().__init__(master, fg_color='transparent')
```

```
        ctk.CTkLabel(self, text='Dashboard',
            font=('Arial', 26, 'bold')).pack(pady=(40, 10))
        ctk.CTkLabel(self, text='Login realizado com sucesso!',
            text_color='#2ECC71').pack(pady=5)

        tabs = ctk.CTkTabview(self, width=480, height=250)
        tabs.pack(pady=20)
        tabs.add('Perfil')
        tabs.add('Estatísticas')
        ctk.CTkLabel(tabs.tab('Perfil'),
            text='admin@email.com').pack(pady=20)
        ctk.CTkLabel(tabs.tab('Estatísticas'),
            text='100 acessos este mês').pack(pady=20)

        ctk.CTkButton(self, text='Sair', width=200,
            fg_color='#C0392B', hover_color='#922B21',
            command=lambda: master.mostrar('login')).pack(pady=10)


# ── App Principal ──────────────────────────────────────────────────
class App(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title('Meu App')
        self.geometry('540x580')
        self.resizable(False, False)

        self.telas = {
            'login':     TelaLogin(self),
            'cadastro':  TelaCadastro(self),
            'dashboard': TelaDashboard(self),
        }
        self.mostrar('login')

    def mostrar(self, nome):
        for tela in self.telas.values():
            tela.pack_forget()
        self.telas[nome].pack(fill='both', expand=True)


App().mainloop()
```

Como testar o login

Use o e-mail 'admin@email.com' e a senha '1234' para entrar no Dashboard. Ou clique em 'Criar conta' para testar o fluxo de cadastro.

Referência Rápida — Métodos Comuns

Widget	Ler valor	Definir valor	Atualizar prop.
CTkEntry	.get()	.insert(0, val)	.configure(text='')
CTkTextbox	.get('1.0','end')	.insert('end', val)	.configure(state='disabled')
CTkCheckBox	.get() → 0/1	.select() / .deselect()	.configure(text='novo')
CTkSwitch	.get() → 0/1	.select() / .deselect()	.configure(text='novo')
CTkSlider	.get()	.set(valor)	.configure(from_=0, to=200)
CTkComboBox	.get()	.set(valor)	.configure(values=[...])
CTkOptionMenu	.get()	.set(valor)	.configure(values=[...])
CTkSegmented	.get()	.set(valor)	.configure(values=[...])
CTkProgressBar	.get()	.set(0.0 a 1.0)	.configure(mode='indeterminate')
CTkLabel	—	—	.configure(text='novo')
CTkButton	—	—	.configure(state='disabled')

Documentação oficial: github.com/TomSchimansky/CustomTkinter

Bons estudos e bons projetos!